

Software Requirements Specification

AI-Driven Autonomous Infrastructure Manager

IEEE 29148-style SRS report

Document Version	1.1
Prepared For	University Software Engineering Project
Prepared By	23K-0668, 23K-0904, 23K-0708
Date	2026-05-04
Standard Basis	ISO/IEC/IEEE 29148:2018 style SRS structure

Academic Project Submission Document

Contents

Software Requirements Specification (SRS)

AI-Driven Autonomous Infrastructure Manager

Revision History

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Intended Audience
 - 1.4 Definitions, Acronyms, and Abbreviations
 - 1.5 References
 - 1.6 Document Overview
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
3. External Interface Requirements
 - 3.1 User Interfaces
 - 3.2 Software Interfaces
 - 3.3 Communication Interfaces
 - 3.4 Hardware Interfaces
4. System Features and Functional Requirements
 - 4.1 Feature A: Backend Request Processing
 - 4.2 Feature B: Gateway-Based Request Routing
 - 4.3 Feature C: ML-Based Health Scoring and Anomaly Detection
 - 4.4 Feature D: Health-Aware Routing Synchronization
 - 4.5 Feature E: Dashboard Monitoring and Visualization
 - 4.6 Feature F: Live Demo Traffic Control
 - 4.7 Feature G: Chaos Engineering
 - 4.8 Feature H: Autoscaling
 - 4.9 Feature I: Audit Logging
5. Non-Functional Requirements
 - 5.1 Performance Requirements
 - 5.2 Reliability and Availability Requirements
 - 5.3 Scalability Requirements
 - 5.4 Security Requirements
 - 5.5 Maintainability Requirements
 - 5.6 Portability Requirements
 - 5.7 Resource Constraints
6. Supporting Information
 - 6.1 Primary Actors
 - 6.2 Use Case Summary
 - 6.3 Out of Scope
 - 6.4 Known Project-Level Limitations
 - 6.5 Acceptance Summary

Software Requirements Specification (SRS)

AI-Driven Autonomous Infrastructure Manager

Revision History

Version	Date	Description
1.0	2026-05-04	Initial IEEE-style SRS derived from the current project implementation
1.1	2026-05-04	Updated chaos and AI-routing demo requirements to reflect backend degradation workflow

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for the AI-Driven Autonomous Infrastructure Manager. The system is an academic demonstration platform for intelligent load balancing, health-aware routing, live observability, chaos injection, and automatic scaling of containerized backend services.

This SRS is intended for:

- instructors and evaluators reviewing the project
- developers maintaining or extending the system
- testers validating implemented behavior
- project stakeholders using the dashboard for live demonstrations

1.2 Scope

The product is a Docker-based microservices system that:

- runs multiple backend application servers
- routes requests through a gateway using multiple balancing strategies
- predicts backend health scores using a weighted ML-style model
- detects anomalous server behavior
- provides a live dashboard for monitoring and demo control
- supports traffic simulation and chaos testing
- attempts dynamic backend scaling using Docker

The product is primarily intended for educational demonstration, experimentation, and requirements engineering coursework rather than production deployment.

1.3 Intended Audience

- Software engineering students
- course instructors
- project demonstrators
- developers working on distributed systems and observability concepts

1.4 Definitions, Acronyms, and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface
ML	Machine Learning
RPS	Requests Per Second
LB	Load Balancer
Docker	Containerization platform used to deploy services
Chaos Injection	Intentional degradation or failure introduced to test resilience
Health Score	Numeric estimate of backend health from 0.0 to 1.0

1.5 References

1. ISO/IEC/IEEE 29148:2018, Systems and software engineering - Life cycle processes - Requirements engineering. Reference page: ISO/IEC/IEEE 29148:2018 (<https://www.iso.org/standard/72089.html>)
2. IEEE Standards Association reference page for 29148. Reference page: IEEE SA 29148 (<https://standards.ieee.org/ieee/29148/12262>)
3. Project README and source implementation in this repository.

Note: IEEE 830 is the older SRS reference model commonly taught in universities; ISO/IEC/IEEE 29148 is the newer requirements engineering standard that superseded it.

1.6 Document Overview

Section 2 describes the product context and constraints. Section 3 defines interface requirements. Section 4 defines system features and functional requirements. Section 5 defines non-functional requirements. Section 6 provides supporting appendices and assumptions.

2. Overall Description

2.1 Product Perspective

The product is a distributed microservices application orchestrated with Docker Compose. It consists of:

- three initial backend services
- one gateway service for routing
- one ML service for health scoring and anomaly analysis
- one orchestrator service for live demo control
- one dashboard service for visualization and interaction
- one autoscaler service for container scaling logic

The system is self-contained and runs on a single machine using Docker Desktop.

2.2 Product Functions

At a high level, the system provides the following capabilities:

- process client requests through backend instances
- distribute requests using round_robin, least_connections, ai_powered, and off modes
- collect and display operational statistics
- compute backend health scores from metrics
- display anomaly signals for backend behavior
- simulate traffic patterns such as constant, ramp, spike, sine_wave, and burst
- inject backend degradation for resilience and AI-routing demonstration
- maintain a recent audit trail of demo actions
- attempt automatic scaling using Docker container control

2.3 User Classes and Characteristics

User Class	Description	Skill Level
Demonstrator	Uses the dashboard during a live presentation	Basic to intermediate
Developer	Extends services, APIs, and Docker deployment	Intermediate to advanced
Tester	Verifies routing, metrics, and demo scenarios	Intermediate
Instructor/Evaluator	Reviews documentation and observes system behavior	Basic to intermediate

2.4 Operating Environment

The system shall operate in the following environment:

- Windows host with Docker Desktop and WSL 2 enabled
- Docker Compose for multi-container orchestration
- Python 3.11-based service containers
- Browser access to the Streamlit dashboard on localhost

Default service exposure in the current implementation:

- Dashboard: localhost:8501
- Gateway: localhost:8000
- Backend containers: localhost:8001, 8002, 8003
- ML service: localhost:8004
- Orchestrator: localhost:8005

2.5 Design and Implementation Constraints

- The system shall be containerized with Docker.
- Service composition shall be defined in Docker Compose.
- Backend services shall be implemented with FastAPI.
- The dashboard shall be implemented with Streamlit.
- The ML service and autoscaler shall use Python.
- Initial deployment shall start with three backend services.
- The project is constrained by local machine resources and Docker Desktop limits.

2.6 User Documentation

The system shall provide:

- a README with installation and startup instructions
- API documentation from framework-generated endpoints where available
- this SRS for formal software requirements

2.7 Assumptions and Dependencies

- Docker Desktop is installed and running.
- The user has permission to run local containers.
- The host machine has browser access to localhost.
- Several parts of the system are intentionally simulated for demo purposes, including some traffic metrics, memory estimates, and dashboard forecasts.
- This document specifies the intended software behavior based on the current academic implementation, not a production SLA.

3. External Interface Requirements

3.1 User Interfaces

3.1.1 Streamlit Dashboard

The dashboard shall provide:

- a system overview showing active strategy, latency, active servers, and request count
- performance charts for latency and throughput
- a server health panel showing CPU and ML health score
- traffic distribution and strategy usage visualizations
- anomaly and forecast panels
- a sidebar command center with Presets, Manual, and Settings tabs

3.1.2 Preset Controls

The dashboard shall provide one-click demo actions for:

- Black Friday Rush
- DDoS Attack
- Normal Operations
- Sine Wave

3.1.3 Manual Controls

The dashboard shall allow the user to:

- select a routing strategy
- set traffic intensity
- choose a traffic pattern
- inject chaos

3.2 Software Interfaces

Interface	Description
Backend to Gateway	Backend services are invoked by the gateway over HTTP
Gateway to ML Service	Gateway fetches health scores from ML service over HTTP
Dashboard to Gateway	Dashboard reads server and routing statistics
Dashboard to Orchestrator	Dashboard sets traffic, strategy, and chaos actions
Dashboard to ML Service	Dashboard reads scores and anomaly data
Autoscaler to Docker Engine	Autoscaler queries and manages backend containers
Orchestrator to Backend Services	Orchestrator triggers backend degradation and recovery over HTTP
Orchestrator to Docker Engine	Orchestrator may use Docker for container control where host access is available

3.3 Communication Interfaces

- Internal service-to-service communication shall use HTTP over the Docker bridge network.
- Host-to-service access shall use mapped localhost ports.
- Request and response payloads shall use JSON for API communication where applicable.

3.4 Hardware Interfaces

No custom hardware interface is required. The only required host capability is support for Docker Desktop and browser-based localhost access.

4. System Features and Functional Requirements

4.1 Feature A: Backend Request Processing

Description: Backend services simulate request handling and expose health and metrics endpoints.

Functional Requirements

- FR-BE-001: The system shall start with three backend service instances.
- FR-BE-002: Each backend service shall expose a GET /health endpoint returning service health and server identity.
- FR-BE-003: Each backend service shall expose a POST /process endpoint that accepts a JSON request payload.
- FR-BE-004: The POST /process endpoint shall return a JSON response indicating whether the request was processed, the backend identity, and the simulated processing time.
- FR-BE-005: Each backend service shall expose a GET /metrics endpoint that returns runtime metrics including CPU usage, memory usage, response time, active connections, and uptime.

4.2 Feature B: Gateway-Based Request Routing

Description: The gateway receives requests and routes them to backend services according to a selected balancing strategy.

Functional Requirements

- FR-GW-001: The gateway shall maintain a registry of backend servers available for routing.
- FR-GW-002: The gateway shall expose a POST /route endpoint that forwards requests to a selected backend server.
- FR-GW-003: The gateway shall support the routing strategies round_robin, least_connections, ai_powered, and off.
- FR-GW-004: Under round_robin, the gateway shall select healthy backend servers in cyclic order.
- FR-GW-005: Under least_connections, the gateway shall route to the healthy backend server with the fewest active connections.
- FR-GW-006: Under ai_powered, the gateway shall select among healthy backend servers using health-score-weighted routing.
- FR-GW-007: Under off, the gateway shall route all traffic to a single healthy backend server until it becomes unavailable.
- FR-GW-008: The gateway shall expose a GET /servers endpoint returning registered server details and health status.
- FR-GW-009: The gateway shall expose a POST /servers/register endpoint for dynamic backend registration.
- FR-GW-010: The gateway shall expose a DELETE /servers/{server_id} endpoint for backend removal.
- FR-GW-011: The gateway shall expose a GET /stats endpoint returning per-strategy usage, per-server routing statistics, and average latency.
- FR-GW-012: The gateway shall expose a GET /health endpoint indicating overall gateway health and number of available servers.
- FR-GW-013: The gateway shall expose a GET /generate-load endpoint that generates routed load against backend /process endpoints for live demonstrations.
- FR-GW-014: If no backend server is healthy, the gateway shall return an error response rather than silently dropping the request.
- FR-GW-015: When a backend request fails, the gateway shall decrement active connection tracking and mark the affected backend unhealthy.

4.3 Feature C: ML-Based Health Scoring and Anomaly Detection

Description: The ML service collects metrics, computes backend health scores, and detects anomalies.

Functional Requirements

- FR-ML-001: The ML service shall collect gateway routing statistics and orchestrator metrics over HTTP.
- FR-ML-002: The ML service shall perform health checks against each backend service.
- FR-ML-003: The ML service shall compute a health score for each backend service in the range 0.0 to 1.0.
- FR-ML-004: Health score computation shall consider CPU usage, memory usage, active connections, error rate, success rate, and latency.
- FR-ML-005: The ML service shall expose a GET /scores endpoint returning health scores for all known backend services.
- FR-ML-006: The ML service shall expose a GET /scores/{server} endpoint returning the health score and metrics for a specific backend.
- FR-ML-007: The ML service shall expose a GET /metrics endpoint returning collected metrics for all monitored backends.
- FR-ML-008: The ML service shall expose a GET /info endpoint describing the model type, features, weights, and thresholds.
- FR-ML-009: The ML service shall retain a bounded history window of metrics for each backend.
- FR-ML-010: The ML service shall expose anomaly detection results through a GET /anomalies endpoint.
- FR-ML-011: The anomaly detector shall use accumulated baseline data before reporting fitted anomaly results.

4.4 Feature D: Health-Aware Routing Synchronization

Description: The gateway periodically consumes health scores from the ML service.

Functional Requirements

- FR-SYNC-001: The gateway shall periodically request the current health scores from the ML service.
- FR-SYNC-002: The gateway shall update internal backend health scores using the values received from the ML service.
- FR-SYNC-003: If score synchronization fails temporarily, the gateway shall continue operating with its last known state.

4.5 Feature E: Dashboard Monitoring and Visualization

Description: The dashboard provides a single pane of glass for system state and demo interaction.

Functional Requirements

- FR-DB-001: The dashboard shall retrieve gateway statistics, ML data, and orchestrator state through HTTP APIs.
- FR-DB-002: The dashboard shall display the current routing strategy, estimated latency, active server count, and total routed requests.
- FR-DB-003: The dashboard shall maintain rolling performance history for charting recent behavior.
- FR-DB-004: The dashboard shall display per-server CPU estimates, request counts, active connection counts, and ML health scores.
- FR-DB-005: The dashboard shall visualize traffic distribution across servers.
- FR-DB-006: The dashboard shall visualize usage distribution across routing strategies.
- FR-DB-007: The dashboard shall display anomaly status and a forecast panel for demo purposes.
- FR-DB-008: The dashboard shall support automatic refresh while preserving the current session state.
- FR-DB-009: The dashboard shall display recent audit events recorded by the orchestrator.

4.6 Feature F: Live Demo Traffic Control

Description: The system provides live traffic simulation to exercise routing, monitoring, and resilience flows.

Functional Requirements

- FR-CTRL-001: The dashboard shall allow users to set traffic pattern and intensity through the orchestrator.
- FR-CTRL-002: The dashboard shall allow users to change the active routing strategy through the orchestrator.
- FR-CTRL-003: The dashboard shall support manual traffic patterns constant, ramp, spike, sine_wave, and burst.
- FR-CTRL-004: The dashboard shall support traffic intensity selection from 10 to 10000 requests per second.
- FR-CTRL-005: The dashboard shall implement a background traffic generator that repeatedly invokes the gateway load-generation endpoint when traffic is active.
- FR-CTRL-006: The system shall provide preset demo scenarios for Black Friday Rush, DDoS Attack, Normal Operations, and Sine Wave.
- FR-CTRL-007: The orchestrator shall expose a GET /api/status endpoint returning current traffic pattern, intensity, strategy, last chaos event, and degraded backend state.
- FR-CTRL-008: The orchestrator shall expose a POST /api/set_traffic endpoint that validates and stores traffic pattern and intensity.
- FR-CTRL-009: The orchestrator shall expose a POST /api/set_strategy endpoint that validates and stores the routing strategy.
- FR-CTRL-010: The orchestrator shall expose a GET /api/metrics endpoint returning metrics used by the dashboard to visualize load state.

4.7 Feature G: Chaos Engineering

Description: The system supports controlled backend degradation for resilience and health-aware routing demonstrations.

Functional Requirements

- FR-CH-001: The orchestrator shall expose a POST /api/inject_chaos endpoint.
- FR-CH-002: Chaos injection shall degrade a selected backend so it remains reachable but responds with higher latency and degraded metrics.
- FR-CH-003: Degraded backends shall expose recovery behavior so normal operation can be restored during the demo.
- FR-CH-004: The orchestrator shall record the most recent chaos event and currently degraded backend in its state.
- FR-CH-005: The orchestrator shall add chaos and recovery events to the audit log.

4.8 Feature H: Autoscaling

Description: The autoscaler monitors backend utilization and attempts to scale backend containers.

Functional Requirements

- FR-AS-001: The autoscaler shall monitor running backend containers through the Docker engine.
- FR-AS-002: The autoscaler shall calculate average CPU utilization across running backend containers.
- FR-AS-003: The autoscaler shall support scaling within the configured bounds of minimum and maximum backend instances.
- FR-AS-004: If average CPU rises above the scale-up threshold and cooldown constraints are satisfied, the autoscaler shall attempt to start an additional backend container.
- FR-AS-005: If average CPU falls below the scale-down threshold and cooldown constraints are satisfied, the autoscaler shall attempt to remove one backend container.

- FR-AS-006: The autoscaler shall expose health and status endpoints for inspection.
- FR-AS-007: The autoscaler shall maintain internal state including last scaling action, current server count, average CPU, and total scale events.

4.9 Feature I: Audit Logging

Description: The orchestrator maintains an in-memory record of recent control-plane actions.

Functional Requirements

- FR-AUD-001: The orchestrator shall record traffic changes in an event log.
- FR-AUD-002: The orchestrator shall record routing strategy changes in an event log.
- FR-AUD-003: The orchestrator shall record chaos injection events in an event log.
- FR-AUD-004: The orchestrator shall expose a GET /api/events endpoint returning recent events.
- FR-AUD-005: The event log shall retain a bounded recent history rather than unbounded growth.

5. Non-Functional Requirements

5.1 Performance Requirements

- NFR-PERF-001: The system shall support a configurable demo traffic intensity range of 10 to 10000 RPS.
- NFR-PERF-002: Each backend service shall simulate request processing in approximately 50 ms to 200 ms under normal operation.
- NFR-PERF-003: The gateway shall enforce finite network timeouts when calling dependent services.
- NFR-PERF-004: The dashboard shall refresh periodically with a default interval of 3 seconds unless changed by the user.

5.2 Reliability and Availability Requirements

- NFR-REL-001: The gateway shall continue operating when the ML service is temporarily unavailable by using the current internal routing state.
- NFR-REL-002: The dashboard shall degrade gracefully when one or more dependent services are unavailable.
- NFR-REL-003: The system shall return explicit error responses when required backend services are unavailable.

5.3 Scalability Requirements

- NFR-SCL-001: The system shall initialize with three backend instances.
- NFR-SCL-002: The autoscaling configuration shall preserve a minimum demo baseline of 3 backend instances and support up to 10 backend instances.
- NFR-SCL-003: Scaling actions shall observe a cooldown period between consecutive scale operations.

5.4 Security Requirements

- NFR-SEC-001: The system may run entirely on localhost for development and demo use.
- NFR-SEC-002: No authentication or authorization is required for the academic demo environment.
- NFR-SEC-003: The absence of authentication shall be treated as an accepted project limitation, not a production-ready security model.

5.5 Maintainability Requirements

- NFR-MNT-001: Each major capability shall be isolated into its own service module.
- NFR-MNT-002: Container orchestration shall be defined declaratively in docker-compose.yml.
- NFR-MNT-003: Service APIs shall use JSON payloads to reduce coupling and ease debugging.

5.6 Portability Requirements

- NFR-PORT-001: The system shall be deployable on any host environment that supports Docker Desktop or equivalent Docker runtime.
- NFR-PORT-002: Service dependencies shall be packaged into container images to reduce host-side setup complexity.

5.7 Resource Constraints

The current implementation defines the following deployment constraints:

- NFR-RSRC-001: Each initial backend container shall be limited to 0.25 CPU and 128 MB memory.
- NFR-RSRC-002: The gateway container shall be limited to 0.5 CPU and 256 MB memory.
- NFR-RSRC-003: The ML service container shall be limited to 0.5 CPU and 512 MB memory.
- NFR-RSRC-004: The autoscaler container shall be limited to 0.25 CPU and 128 MB memory.
- NFR-RSRC-005: The orchestrator container shall be limited to 0.25 CPU and 256 MB memory.

- NFR-RSRC-006: The dashboard container shall be limited to 0.5 CPU and 512 MB memory.

6. Supporting Information

6.1 Primary Actors

Actor	Goal
Demo User	Observe infrastructure behavior and run predefined scenarios
Developer	Extend or modify services and routing logic
Docker Engine	Provide container runtime and management capabilities

6.2 Use Case Summary

Use Case	Description
UC-01	View system health and metrics on the dashboard
UC-02	Switch routing strategies during live operation
UC-03	Apply manual or preset traffic patterns
UC-04	Generate live routed load through the gateway
UC-05	Inject backend degradation and observe health-aware routing behavior
UC-06	Observe anomaly and health-score changes over time
UC-07	Allow the autoscaler to add or remove backend capacity

6.3 Out of Scope

The following are outside the scope of this academic system:

- production authentication and authorization
- persistent audit storage
- multi-host or cloud-native orchestration such as Kubernetes
- hardened security controls
- formal SLA management
- business-domain transaction processing

6.4 Known Project-Level Limitations

- Some displayed metrics are simulated or simplified for demonstration.
- Forecast visualization is dashboard-generated and is not a production forecasting pipeline.
- Container-kill chaos depends on Docker accessibility from containers and the local host environment; the current demo uses HTTP-based backend degradation for reliable live demonstration.
- This SRS is aligned to the current repository behavior and intended product goals; future implementation changes may require SRS updates.

6.5 Acceptance Summary

The software shall be considered acceptable for the course project when:

- all core services start successfully with Docker Compose
- the dashboard is accessible from localhost
- routing strategies can be changed from the dashboard
- traffic generation changes visible charts and counters
- health scores are available from the ML service
- chaos injection can visibly degrade a backend and recovery can restore normal operation
- system behavior remains observable after traffic or chaos actions